

Rust

Violet

2026-04-13

Chapter 0

目 录

I	Rust 核心基础	
1.	Rust 入门与环境搭建	5
2.	基础语法与数据类型	6
2.1.	变量与可变性	6
2.2.	标量类型	8
2.3.	复合类型	11
2.4.	注释	13
2.5.	格式化输出	16
2.6.	类型转换	20
2.7.	总结	22
3.	所有权与借用机制	23
3.1.	所有权规则	23
3.2.	栈与堆	23
3.3.	移动语义 (Move)	24
3.4.	函数与所有权	26
3.5.	引用与借用	27
3.6.	切片 (Slices)	29
3.7.	所有权实践示例	30
3.8.	常见错误与解决	32
3.9.	总结	33



Rust 核心基础

1. Rust 入门与环境搭建 ·····	5
2. 基础语法与数据类型 ·····	6
3. 所有权与借用机制 ·····	23

Chapter 1

Rust 入门与环境搭建

Chapter 2

基础语法与数据类型

Rust 是一门系统级编程语言，强调安全性、并发性和性能。它没有垃圾回收器，通过所有权系统在编译时保证内存安全。

NOTE

Rust 的核心理念是**零成本抽象**：你只需要为使用的特性付出代价，而且不能比手写的 C/C++ 代码更慢。

1 变量与可变性

Rust 的变量默认是不可变的 (immutable)，这是其安全性的基石之一。

1.1 不可变变量

```
1 fn main() {  
2     let x = 5;  
3     println!("x = {}", x);  
4  
5     // 以下代码会编译错误  
6     // x = 6; // error[E0384]: cannot assign twice to immutable variable `x`  
7 }
```

特点：

- 使用 `let` 关键字声明
- 默认不可变
- 编译器保证不会被意外修改
- 有助于并发安全

TIP

Rust 鼓励不可变性，这可以减少 bug 并提高代码的可读性和可维护性。

1.2 可变变量

使用 `mut` 关键字声明可变变量：

```
1 fn main() {  
2     let mut x = 5;  
3     println!("x = {}", x);  
4  
5     x = 6; // 正确：x是可变的  
6     println!("x = {}", x);  
}
```

```
7 }
```

注意：

- 必须显式使用 `mut`
- 只能在同一作用域内修改
- 可变引用有严格限制（后续章节详述）

1.3 常量

常量使用 `const` 关键字声明，与变量有重要区别：

```
1 const MAX_POINTS: u32 = 100_000;
2 const SECONDS_PER_DAY: u32 = 86_400;
```



常量的特点：

- 必须标注类型
- 只能设置为常量表达式（不能在运行时计算）
- 可以在任何作用域声明，包括全局
- 命名约定：全大写，下划线分隔
- 不能使用 `mut`

常量 vs 不可变变量：

特性	常量 (<code>const</code>)	不可变变量 (<code>let</code>)
关键字	<code>const</code>	<code>let</code>
类型标注	必须	可选（类型推断）
赋值	常量表达式	任意表达式
作用域	全局可用	当前作用域
可变性	永远不可变	可用 <code>mut</code> 变为可变
内存位置	可能嵌入代码	栈上分配

CAUTION

不要混淆常量和不可变变量。常量用于真正的编译时常量，而不可变变量用于运行时的值。

1.4 变量遮蔽（Shadowing）

Rust 允许用相同的名称声明新变量，这会“遮蔽”之前的变量：

```
1 fn main() {
2     let x = 5;
3     let x = x + 1; // 遮蔽了之前的x
4     let x = x * 2; // 再次遮蔽
5
6     println!("x = {}", x); // 输出：x = 12
7 }
```



遮蔽的特点：

- 可以改变类型

- 可以改变可变性
- 之前的变量仍然存在，只是无法访问
- 不同于可变变量，遮蔽创建了一个全新的变量

```
1 fn main() {  
2     let spaces = "  ";  
3     let spaces = spaces.len(); // 从&str变为usize  
4  
5     println!("spaces length = {}", spaces); // 输出: 3  
6 }
```

TIP

遮蔽比可变变量更安全，因为它创建了新值而不是修改旧值。适合转换数据的场景。

2 标量类型

标量类型代表单个值。Rust 有四种基本标量类型。

2.1 整数类型

有符号整数

- `i8`: 8 位，范围 -128 到 127
- `i16`: 16 位，范围 -32,768 到 32,767
- `i32`: 32 位，范围约 -21 亿到 21 亿（默认）
- `i64`: 64 位
- `i128`: 128 位
- `isize`: 指针大小（取决于架构）

无符号整数

- `u8`: 8 位，范围 0 到 255
- `u16`: 16 位
- `u32`: 32 位
- `u64`: 64 位
- `u128`: 128 位
- `usize`: 指针大小（常用于索引）

```
1 fn main() {  
2     let decimal = 98_222; // 十进制  
3     let hex = 0xff; // 十六进制  
4     let octal = 0o77; // 八进制  
5     let binary = 0b1111_0000; // 二进制  
6     let byte = b'A'; // 字节 (u8 only)  
7  
8     println!("decimal: {}, hex: {}, octal: {}, binary: {}, byte: {}",  
9             decimal, hex, octal, binary, byte);  
}
```

```
10 }
```

数字字面量：

- 可以使用下划线提高可读性： `1_000_000`
- 可以指定类型后缀： `57u8` , `3.14f64`

NOTE

`i32` 是 Rust 的默认整数类型，即使在 64 位系统上也是如此。它在大多数情况下提供了性能和范围的平衡。

整数溢出

Rust 在调试模式下检查整数溢出，在发布模式下进行回绕 (wrapping)：

```
1 fn main() {  
2     let x: u8 = 255;  
3     // let y = x + 1; // 调试模式: panic! 发布模式: y = 0  
4 }
```



处理方式：

- `wrapping_add()` : 始终回绕
- `checked_add()` : 返回 `Option`
- `overflowing_add()` : 返回值和溢出标志
- `saturating_add()` : 饱和到最大值

```
1 fn main() {  
2     let x: u8 = 255;  
3  
4     println!("wrapping: {}", x.wrapping_add(1)); // 0  
5     println!("checked: {:?}", x.checked_add(1)); // None  
6     println!("saturating: {}", x.saturating_add(1)); // 255  
7 }
```



2.2 浮点类型

Rust 有两种浮点类型：

- `f32` : 32 位单精度
- `f64` : 64 位双精度 (默认)

```
1 fn main() {  
2     let x = 2.0; // f64  
3     let y: f32 = 3.0; // f32  
4  
5     println!("x = {}, y = {}", x, y);  
6 }
```



浮点运算：

```
1 fn main() {  
2     let sum = 5.0 + 10.0;
```



```
3     let difference = 95.5 - 4.3;
4     let product = 4.0 * 30.0;
5     let quotient = 56.7 / 32.2;
6     let remainder = 43 % 5; // 整数取模
7
8     println!("sum={}, diff={}, prod={}, quot={}, rem={}",
9             sum, difference, product, quotient, remainder);
10 }
```

CAUTION

浮点数不遵循结合律和分配律！`(a + b) + c` 可能不等于 `a + (b + c)`。不要直接用 `==` 比较浮点数。

2.3 布尔类型

布尔类型 `bool` 有两个可能的值：`true` 和 `false`。

```
1 fn main() {
2     let t = true;
3     let f: bool = false;
4
5     if t {
6         println!("t is true");
7     }
8 }
```

布尔运算：

```
1 fn main() {
2     let a = true;
3     let b = false;
4
5     println!("a && b = {}", a && b); // AND
6     println!("a || b = {}", a || b); // OR
7     println!("!a = {}", !a);        // NOT
8 }
```

NOTE

Rust 的布尔类型占用 1 个字节，不是 1 个 bit。这是为了内存对齐和访问效率。

2.4 字符类型

`char` 类型表示一个 Unicode 标量值，占用 4 个字节。

```
1 fn main() {
2     let c = 'z';
3     let z: char = 'Z';
4     let heart_eyed_cat = '😺';
5
6     println!("c={}, z={}, cat={}", c, z, heart_eyed_cat);
7 }
```

特点:

- 用单引号 `'` 包围
- Unicode 标量值 (U+0000 到 U+D7FF, U+E000 到 U+10FFFF)
- 不只是 ASCII, 支持全球语言
- 占用 4 字节 (32 位)

```
1 fn main() {  
2     // 转义字符  
3     let newline = '\n';  
4     let tab = '\t';  
5     let backslash = '\\';  
6     let quote = '\"';  
7  
8     // Unicode转义  
9     let heart = '\u{2764}'; // ❤️  
10  
11     println!("newline, tab, backslash, quote, heart");  
12 }
```

NOTE

`char` 和字符串切片 `&str` 不同。`char` 是单个 Unicode 标量值, 而字符串是 UTF-8 编码的字节序列。

3 复合类型

复合类型可以将多个值组合成一个类型。Rust 有两个原始的复合类型: 元组和数组。

3.1 元组 (Tuple)

元组是将多个类型的值组合成一个复合类型的一般方式。

基本用法

```
1 fn main() {  
2     let tup: (i32, f64, u8) = (500, 6.4, 1);  
3  
4     // 解构元组  
5     let (x, y, z) = tup;  
6     println!("x={}, y={}, z={}", x, y, z);  
7  
8     // 通过索引访问  
9     let five_hundred = tup.0;  
10    let six_point_four = tup.1;  
11    let one = tup.2;  
12 }
```

特点:

- 长度固定
- 可以包含不同类型的值

- 通过模式匹配解构
- 通过点号加索引访问 (`.0`, `.1`, `.2`)

空元组

```
1 fn main() {
2     let unit: () = ();
3     println!("unit tuple: {:?}", unit);
4 }
```



空元组 `()` 也称为单元类型，表示没有值。函数的返回值如果是 `()`，通常省略不写。

元组作为返回值

```
1 fn calculate_stats(numbers: &[i32]) -> (i32, i32, f64) {
2     let min = *numbers.iter().min().unwrap();
3     let max = *numbers.iter().max().unwrap();
4     let avg = numbers.iter().sum::<i32>() as f64 / numbers.len() as f64;
5
6     (min, max, avg)
7 }
8
9 fn main() {
10     let nums = vec![1, 2, 3, 4, 5];
11     let (min, max, avg) = calculate_stats(&nums);
12
13     println!("min={}, max={}, avg={}", min, max, avg);
14 }
```



TIP

当函数需要返回多个值时，元组是一个简单有效的选择。但对于复杂的数据结构，建议使用结构体。

3.2 数组 (Array)

数组是相同类型元素的固定长度集合。

基本用法

```
1 fn main() {
2     let months = ["January", "February", "March",
3                 "April", "May", "June",
4                 "July", "August", "September",
5                 "October", "November", "December"];
6
7     let first = months[0];
8     let last = months[11];
9
10    println!("First: {}, Last: {}", first, last);
11 }
```



特点：

- 长度固定（编译时确定）

- 所有元素类型相同
- 存储在栈上
- 通过索引访问（从 0 开始）

数组类型标注

```
1 fn main() {  
2     let a: [i32; 5] = [1, 2, 3, 4, 5];  
3  
4     // 初始化所有元素为相同值  
5     let b = [3; 5]; // 等同于 [3, 3, 3, 3, 3]  
6  
7     println!("a={:?}", b={:?}", a, b);  
8 }
```



类型格式: `[类型; 长度]`

数组访问与安全

```
1 fn main() {  
2     let a = [1, 2, 3, 4, 5];  
3  
4     let index = 10;  
5     let element = a[index]; // panic: index out of bounds  
6 }
```



CAUTION

Rust 在运行时检查数组访问边界。如果索引超出范围，程序会 panic 而不是访问非法内存。这保证了内存安全，但会带来轻微的性能开销。

数组 vs Vec

特性	数组 [T; N]	Vec<T>
长度	固定（编译时）	动态（运行时）
内存位置	栈	堆
性能	更快	稍慢
灵活性	低	高
适用场景	已知大小	未知或变化大小

TIP

当你确定元素数量不会改变时，使用数组。否则，使用 `Vec<T>`（向量），它是可增长的数组。

4 注释

良好的注释是高质量代码的重要组成部分。

4.1 行注释

使用 `//` 添加行注释：

```
1 fn main() {  
2     // 这是一个行注释  
3     let x = 5; // 这也是注释  
4  
5     // 多行注释  
6     // 可以这样写  
7     // 每一行都用 //  
8 }
```

4.2 块注释

使用 `/* */` 添加块注释：

```
1 fn main() {  
2     /*  
3      * 这是一个块注释  
4      * 可以跨越多行  
5      * 适合较长的说明  
6      */  
7     let x = 5;  
8  
9     /* 也可以这样写短注释 */  
10 }
```

NOTE

Rust 支持嵌套块注释： `/* outer /* inner */ outer */`。这在 C/C++ 中是不支持的。

4.3 文档注释

文档注释以 `///` 或 `///` 开头，会被 `cargo doc` 工具提取生成 HTML 文档。

行文档注释 (///)

用于注释紧随其后的项：

```
1 /// 计算两个数的和  
2 ///  
3 /// # Examples  
4 ///  
5 /// ```  
6 /// let result = add(2, 3);  
7 /// assert_eq!(result, 5);  
8 /// ```  
9 fn add(a: i32, b: i32) -> i32 {  
10     a + b  
11 }
```

块文档注释 (/**/)

```
1  /**
2   * 计算两个数的乘积
3   *
4   * # Arguments
5   *
6   * * `a` - 第一个数
7   * * `b` - 第二个数
8   *
9   * # Returns
10  *
11  * 两数的乘积
12  */
13 fn multiply(a: i32, b: i32) -> i32 {
14     a * b
15 }
```

模块文档注释 (//!)

用于注释包含它的项（通常是模块或 crate）：

```
1  //! # My Crate
2  //!
3  //! `my_crate` is a collection of utilities for performing calculations.
4
5  /// Add two numbers
6 pub fn add(a: i32, b: i32) -> i32 {
7     a + b
8 }
```

4.4 文档注释的 Markdown 特性

文档注释支持 Markdown 格式：

```
1  /// # Summary
2  ///
3  /// A brief description of the function.
4  ///
5  /// # Arguments
6  ///
7  /// * `name` - The name of the person
8  /// * `age` - The age of the person
9  ///
10 /// # Returns
11 ///
12 /// A greeting string
13 ///
14 /// # Examples
15 ///
16 /// ```
17 /// let greeting = greet("Alice", 30);
```

```
18 /// assert_eq!(greeting, "Hello, Alice! You are 30 years old.");
19 /// \\\`
20 ///
21 /// # Panics
22 ///
23 /// Panics if `name` is empty.
24 ///
25 /// # Errors
26 ///
27 /// Returns an error if `age` is negative.
28 ///
29 /// # Safety
30 ///
31 /// This function is safe to call.
32 fn greet(name: &str, age: u32) -> String {
33     format!("Hello, {}! You are {} years old.", name, age)
34 }
```

常用的文档标记:

- `# Examples`: 代码示例 (会被测试)
- `# Panics`: 说明 panic 条件
- `# Errors`: 说明返回的错误
- `# Safety`: 说明不安全操作的前提条件
- `# Arguments`: 参数说明

TIP

经常运行 `cargo doc --open` 来查看生成的文档。良好的文档注释可以自动生成专业的 API 文档。

5 格式化输出

Rust 提供了强大的格式化输出功能, 主要通过 `println!`、`format!` 等宏实现。

5.1 println! & print!

```
1 fn main() {
2     let name = "Alice";
3     let age = 30;
4
5     println!("Hello, {}! You are {} years old.", name, age);
6 }
```



特点:

- 自动添加换行符
- 支持多个占位符 `{}`
- 类型安全 (编译时检查)

```
1 fn main() {
2     print!("This is on the same line");
}
```



```
3 print!(" and this too.\n");
4 }
```

NOTE

区别: `print!` 不自动添加换行符。

5.2 eprintln! 和 eprint!

输出到标准错误流:

```
1 fn main() {
2     eprintln!("Error: something went wrong!");
3 }
```



用途: 错误信息和诊断信息应该输出到 `stderr`。

5.3 格式化占位符

基本占位符

```
1 fn main() {
2     let x = 42;
3
4     // 使用 {} 进行默认格式化
5     println!("Value: {}", x);
6 }
```



常用格式化 trait:

- `Display` (`{}`): 用户友好的输出, 需手动实现
- `Debug` (`{:?}`): 开发者调试输出, 可自动派生
- `Pretty Debug` (`{:#?}`): 美化 Debug 格式, 多行输出

NOTE

在 Typst 文档中, 包含复杂格式化字符串的 Rust 代码可能需要特殊处理。实际使用时请参考 Rust 官方文档。

位置参数

```
1 fn main() {
2     println!("{0} {1} {0}", "hello", "world");
3     // 输出: hello world hello
4 }
```



命名参数

```
1 fn main() {
2     println!("{greeting}, {name}!", greeting = "Hello", name = "Alice");
3 }
```



I 格式化选项

宽度

对齐选项：

- `{:>5}`：右对齐，宽度 5
- `{:<5}`：左对齐，宽度 5
- `{:^5}`：居中，宽度 5
- `{:*^5}`：使用 `*` 填充，居中对齐

```
1 fn main() {  
2     let text = "hi";  
3     println!("Aligned text example");  
4 }
```

精度

```
1 fn main() {  
2     let pi = 3.14159265359;  
3     println!("{:.2}", pi); // 3.14  
4     println!("{:.4}", pi); // 3.1416  
5 }
```

进制

```
1 fn main() {  
2     let x = 42;  
3     println!("Decimal: {}", x); // 42  
4     println!("Binary: {:b}", x); // 101010  
5     println!("Octal: {:o}", x); // 52  
6     println!("Hex: {:x}", x); // 2a  
7     println!("HEX: {:X}", x); // 2A  
8 }
```

指针地址

```
1 fn main() {  
2     let x = 42;  
3     let ptr = &x;  
4     println!("Pointer: {:p}", ptr);  
5 }
```

5.4 format! 宏

`format!` 返回格式化的字符串，而不是直接输出：

```
1 fn main() {  
2     let name = "Alice";  
3     let age = 30;  
4 }
```

```
5 let message = format!("{}", is {} years old", name, age);
6 println!("{}", message);
7 }
```

用途:

- 构建字符串
- 日志记录
- 错误消息

5.5 Display & Debug

Rust 有两个主要的格式化 trait:

Display ({})

面向用户的格式化, 需要手动实现:

```
1 use std::fmt;
2
3 struct Point {
4     x: f64,
5     y: f64,
6 }
7
8 impl fmt::Display for Point {
9     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
10        write!(f, "({}, {})", self.x, self.y)
11    }
12 }
13
14 fn main() {
15     let p = Point { x: 1.0, y: 2.0 };
16     // 使用 {} 占位符
17     println!("Point: {}", p);
18 }
```



Debug (:{?})

面向开发者的调试格式化, 可以自动派生:

```
1 #[derive(Debug)]
2 struct Point {
3     x: f64,
4     y: f64,
5 }
6
7 fn main() {
8     let p = Point { x: 1.0, y: 2.0 };
9     // 使用 {:?} 进行Debug格式化
10    println!("Debug output available");
11 }
```



TIP

对于自定义类型，优先实现 `Debug`（使用 `#[derive(Debug)]`）。如果需要用户友好的输出，再实现 `Display`。

5.6 常见格式化示例

```
1 fn main() {
2     // 百分比
3     let percentage = 0.95 \* 100.0;
4     println!("Percentage: {:.1}%", percentage);
5
6     // 科学计数法
7     println!("Scientific: {:e}", 1000000.0);
8
9     // 填充零
10    println!("Zero padded: {:0>5}", 42);
11
12    // 截断字符串
13    println!("Truncated: {:.3}", "hello");
14
15    // 布尔值
16    println!("{}", true);
17    println!("{}", false);
18 }
```

6 类型转换

Rust 提供了多种类型转换机制。

6.1 隐式转换（Coercion）

某些情况下 Rust 会自动转换类型：

```
1 fn main() {
2     let x: i32 = 5;
3     let y: i64 = x as i64; // 需要显式转换
4 }
```

CAUTION

Rust 几乎没有隐式数值类型转换。这与 C/C++ 不同，有助于避免意外的精度丢失。

6.2 显式转换（as）

使用 `as` 关键字进行基本类型转换：

```
1 fn main() {
2     let decimal = 65.4321_f32;
3 }
```

```

4    // 转换为整数（截断）
5    let integer = decimal as u8;
6    println!("{}", decimal, integer); // 65.4321 -> 65
7
8    // 字符转换
9    let character = integer as char;
10   println!("{}", integer, character); // 65 -> A
11
12   // 不同类型整数转换
13   let a: u8 = 200;
14   let b: u16 = a as u16;
15   let c: i32 = a as i32;
16 }

```

注意事项：

- 浮点转整数会截断小数部分
- 大值转小类型会溢出（回绕）
- 负数转无符号类型会产生大正数

```

1 fn main() {
2     let large_number: i32 = 1000;
3     let small_number = large_number as u8; // 溢出: 1000 % 256 = 232
4     println!("{}", large_number, small_number);
5 }

```



6.3 TryFrom 和 TryInto

安全的转换方式，返回 `Result`：

```

1 use std::convert::TryFrom;
2 use std::convert::TryInto;
3
4 fn main() {
5     let big_number: i64 = 1000;
6
7     // TryFrom - 尝试转换
8     let result = u8::try_from(big_number);
9
10    match result {
11        Ok(value) => println!("Success: {}", value),
12        Err(_) => println!("Conversion failed"),
13    }
14
15    // TryInto - 另一种方式
16    let converted: Result<u8, _> = big_number.try_into();
17
18    if let Ok(value) = converted {
19        println!("Converted: {}", value);
20    }
21 }

```



TIP

优先使用 `TryFrom` / `TryInto` 进行可能失败的转换，它们更安全且符合 Rust 的错误处理哲学。

6.4 From 和 Into

无失败的转换，自动实现：

```
1 fn main() {  
2     let my_str = "hello";  
3  
4     // Into  
5     let my_string: String = my_str.into();  
6  
7     // From (等价)  
8     let my_string2 = String::from(my_str);  
9 }
```

规则：如果实现了 `From<T> for U`，则自动实现 `Into<U> for T`。

7 总结

本节介绍了 Rust 的基础语法和数据类型：

- 变量与可变性：默认不可变，使用 `mut` 声明可变变量，`const` 声明常量
- 标量类型：整数、浮点数、布尔、字符
- 复合类型：元组和数组
- 注释：行注释、块注释、文档注释
- 格式化输出：`println!`、`format!`、格式化选项
- 类型转换：`as`、`TryFrom`、`From`

下一节将深入探讨 Rust 最核心的概念：所有权与借用机制。

Chapter 3

所有权与借用机制

所有权（Ownership）是 Rust 最独特和最重要的特性，它使 Rust 能够在没有垃圾回收器的情况下保证内存安全。理解所有权是掌握 Rust 的关键。

NOTE

所有权的规则在编译时检查，因此不会带来运行时开销。这是 Rust 零成本抽象的核心体现。

1 所有权规则

Rust 的所有权系统基于三条简单但强大的规则：

1. Rust 中的每个值都有一个变量，称为其所有者
2. 一次只能有一个所有者
3. 当所有者离开作用域时，值将被丢弃

```
1 fn main() {  
2     // 规则1: s是String值的所有者  
3     let s = String::from("hello");  
4  
5     // 规则2: s是唯一的所有者  
6     // let t = s; // 如果这样做, s不再有效  
7  
8     // 规则3: 当main函数结束时, s被丢弃  
9 } // s在这里离开作用域, 内存被释放
```



2 栈与堆

理解所有权需要先了解内存管理的基础：栈和堆。

2.1 栈 (Stack)

特点：

- 后进先出（LIFO）
- 所有数据必须有已知的大小
- 分配和释放非常快
- 存储在栈上的数据是“拥有”的

```
1 fn main() {  
2     // 这些类型大小已知, 存储在栈上
```



```

3  let x = 5;           // i32, 4字节
4  let y = true;        // bool, 1字节
5  let z = (1, 2);       // (i32, i32), 8字节
6 } // 按相反顺序释放: z, y, x

```

2.2 堆 (Heap)

特点:

- 组织性较差
- 分配时需要查找足够大的空间
- 比栈慢
- 需要手动管理或使用垃圾回收

```

1 fn main() {
2     // String数据存储在堆上
3     let s = String::from("hello");
4     // s本身在栈上 (指针、长度、容量)
5     // 实际字符串数据在堆上
6 }

```



2.3 栈 vs 堆对比

特性	栈	堆
速度	快	慢
大小	固定、已知	动态、未知
访问	直接	通过指针
管理	自动	需要跟踪
典型类型	整数、布尔、元组	String、Vec、Box

TIP

Rust 通过所有权系统在编译时跟踪堆上数据的生命周期，无需垃圾回收器。

3 移动语义 (Move)

当一个值被赋值给另一个变量或传递给函数时，所有权会转移。

3.1 基本移动

```

1 fn main() {
2     let s1 = String::from("hello");
3     let s2 = s1; // s1的所有权移动到s2
4
5     // println!("{}", s1); // 错误! s1不再有效
6     println!("{}", s2); // 正确
7 } // 只有s2被释放, s1已经无效

```



发生了什么：

```
1 Before: s1 -> [heap data "hello"]
2 After:  s2 -> [heap data "hello"]
3       s1 -> (invalid)
```

CAUTION

这与 C++ 不同！C++ 会进行深拷贝，而 Rust 只复制栈上的指针、长度和容量，不复制堆上的数据。这避免了双重释放问题。

3.2 为什么不是拷贝？

对于复杂类型（如 String），Rust 选择移动而非拷贝的原因：

1. 性能：避免昂贵的堆内存复制
2. 安全：防止双重释放（double free）
3. 明确：清楚地表明所有权转移

```
1 fn main() {
2     let s1 = String::from("hello");
3     let s2 = s1.clone(); // 显式克隆，深拷贝
4
5     println!("s1={}, s2={}", s1, s2); // 都有效
6 }
```

TIP

如果需要保留原值，使用 `.clone()` 方法进行深拷贝。但要注意性能开销。

3.3 仅限栈的类型（Copy trait）

某些简单类型实现了 `Copy` trait，它们的行为不同：

```
1 fn main() {
2     let x = 5;
3     let y = x; // x 仍然有效，因为 i32 实现了 Copy
4
5     println!("x={}, y={}", x, y); // 都有效
6 }
```

实现 Copy 的类型：

- 所有整数类型（`i32`，`u64` 等）
- 布尔类型（`bool`）
- 浮点类型（`f64`，`f32`）
- 字符类型（`char`）
- 元组（仅当所有元素都实现 Copy 时）

不实现 Copy 的类型：

- `String`
- `Vec<T>`
- `Box<T>`

- 任何包含上述类型的结构体

NOTE

Copy 和 Move 的区别：Copy 类型赋值后原值仍有效，Move 类型赋值后原值失效。

4 函数与所有权

将值传递给函数也会发生移动或拷贝。

4.1 参数传递

```
1 fn main() {
2     let s = String::from("hello");
3
4     takes_ownership(s); // s的所有权移动到函数中
5
6     // println!("{}", s); // 错误! s不再有效
7
8     let x = 5;
9
10    makes_copy(x); // x被拷贝到函数中
11
12    println!("{}", x); // 正确, x仍然有效
13 }
14
15 fn takes_ownership(some_string: String) {
16     println!("{}", some_string);
17 } // some_string在这里离开作用域并被释放
18
19 fn makes_copy(some_integer: i32) {
20     println!("{}", some_integer);
21 } // some_integer是Copy类型, 不影响原值
```

4.2 返回值与所有权

函数也可以返回所有权：

```
1 fn main() {
2     let s1 = gives_ownership(); // 接收所有权
3
4     let s2 = String::from("hello");
5     let s3 = takes_and_gives_back(s2); // s2移动进去, s3接收返回
6
7     // println!("{}", s2); // 错误! s2已移动
8     println!("s1={}, s3={}", s1, s3);
9 }
10
11 fn gives_ownership() -> String {
12     let some_string = String::from("yours");
```

```
13     some_string // 返回，所有权转移到调用者
14 }
15
16 fn takes_and_gives_back(a_string: String) -> String {
17     a_string // 返回，所有权转移回调用者
18 }
```

TIP

这种模式很繁琐！每次都需要转移和返回所有权。这就是为什么我们需要引用和借用。

5 引用与借用

引用允许你使用值而不获取所有权。

5.1 不可变引用

```
1 fn main() {
2     let s1 = String::from("hello");
3
4     let len = calculate_length(&s1); // 借用s1，不获取所有权
5
6     println!("The length of '{}' is {}.", s1, len); // s1仍然有效
7 }
8
9 fn calculate_length(s: &String) -> usize { // s是对String的引用
10     s.len()
11 } // s离开作用域，但它不拥有数据，所以不释放
```

关键概念：

- `&s1`：创建对 `s1` 的引用
- `&String`：参数类型是 `String` 的引用
- 借用（Borrowing）：临时使用值，不获取所有权

5.2 引用规则

Rust 对引用有严格的规则：

规则 1：可变引用限制

在同一时间，你只能有一个可变引用：

```
1 fn main() {
2     let mut s = String::from("hello");
3
4     let r1 = &mut s;
5     // let r2 = &mut s; // 错误！不能同时有多个可变引用
6
7     r1.push_str(", world");
8     println!("{}", r1);
```

```
9 } // r1在这里离开作用域，可以创建新的可变引用
```

CAUTION

这个限制防止了数据竞争（data race）。数据竞争会导致未定义行为，是许多 bug 的根源。

规则 2：不可变和可变引用不能共存

当你有不可变引用时，不能有可变引用：

```
1 fn main() {  
2     let mut s = String::from("hello");  
3  
4     let r1 = &s; // 没问题  
5     let r2 = &s; // 没问题，多个不可变引用可以共存  
6     // let r3 = &mut s; // 错误！不能在有不可变引用时创建可变引用  
7  
8     println!("{}", r1, r2);  
9     // r1和r2不再使用后，可以创建可变引用  
10    let r3 = &mut s; // 没问题  
11    println!("{}", r3);  
12 }
```



原因：如果有不可变引用正在读取数据，同时有可变引用在修改数据，会导致数据不一致。

规则 3：引用必须始终有效

dangling reference（悬垂引用）是不允许的：

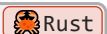
```
1 // fn dangle() -> &String { // 错误！返回悬垂引用  
2 //     let s = String::from("hello");  
3 //     &s  
4 // } // s在这里离开作用域并被释放，返回的引用指向无效内存  
5  
6 fn no_dangle() -> String {  
7     let s = String::from("hello");  
8     s // 返回所有权，而不是引用  
9 }
```

**NOTE**

Rust 的借用检查器在编译时确保引用始终有效，防止了悬垂引用问题。

5.3 可变引用

```
1 fn main() {  
2     let mut s = String::from("hello");  
3  
4     change(&mut s);  
5  
6     println!("{}", s); // 输出: hello, world  
7 }  
8  
9 fn change(some_string: &mut String) {
```



```
10     some_string.push_str(", world");
11 }
```

注意:

- `let mut s`: `s` 必须是可变的
- `&mut s`: 创建可变引用
- `&mut String`: 参数类型是可变引用

6 切片 (Slices)

切片是一种特殊的引用，它不拥有数据，而是引用集合的一部分。

6.1 字符串切片

```
1 fn main() {
2     let s = String::from("hello world");
3
4     let hello = &s[0..5]; // "hello"
5     let world = &s[6..11]; // "world"
6
7     println!("{}", hello, world);
8 }
```

 Rust

语法: `&s[start..end]`

- `start`: 起始索引 (包含)
- `end`: 结束索引 (不包含)

简化语法

```
1 fn main() {
2     let s = String::from("hello world");
3
4     let slice1 = &s[0..2]; // "he"
5     let slice2 = &s[..2]; // "he" (省略起始, 默认为0)
6
7     let slice3 = &s[3..5]; // "lo"
8     let slice4 = &s[3..]; // "lo world" (省略结束, 默认为末尾)
9
10    let slice5 = &s[..]; // 整个字符串
11 }
```

 Rust

字符串切片的类型

字符串切片的类型是 `&str`:

```
1 fn main() {
2     let s = String::from("hello world");
3     let word = first_word(&s);
4 }
```

 Rust

```

5     println!("First word: {}", word);
6 }
7
8 fn first_word(s: &String) -> &str {
9     let bytes = s.as_bytes();
10
11     for (i, &item) in bytes.iter().enumerate() {
12         if item == b' ' {
13             return &s[0..i];
14         }
15     }
16
17     &s[..]
18 }

```

TIP

推荐使用 `&str` 作为函数参数类型，而不是 `&String`。这样函数可以接受 String 切片和字符串字面量。

字符串字面量就是切片

```

1 fn main() {
2     let s = "Hello, world!"; // s的类型是 &str
3
4     // 字符串字面量直接存储在程序的二进制文件中
5     // 它是不可变的，并且是切片类型
6 }

```



6.2 数组切片

```

1 fn main() {
2     let a = [1, 2, 3, 4, 5];
3
4     let slice = &a[1..3]; // &[i32], 包含 [2, 3]
5
6     println!("{:?}", slice);
7 }
8
9 fn first_element(arr: &[i32]) -> Option<&i32> {
10     arr.get(0)
11 }

```



数组切片的类型: `&[T]`

7 所有权实践示例

7.1 示例 1: 处理用户输入

```

1 fn main() {
2     let input = String::from(" hello world ");

```



```
3
4 // 不需要所有权，使用引用
5 let trimmed = trim_whitespace(&input);
6 println!("Trimmed: '{}'", trimmed);
7
8 // 仍然可以使用input
9 println!("Original: '{}'", input);
10 }
11
12 fn trim_whitespace(s: &str) -> &str {
13     s.trim()
14 }
```

7.2 示例 2：构建字符串

```
1 fn main() {
2     let mut result = String::new();
3
4     append_greeting(&mut result, "Alice");
5     append_greeting(&mut result, "Bob");
6
7     println!("{}", result);
8 }
9
10 fn append_greeting(buffer: &mut String, name: &str) {
11     buffer.push_str("Hello, ");
12     buffer.push_str(name);
13     buffer.push('\n');
14 }
```



7.3 示例 3：返回数据

```
1 fn main() {
2     // 方式1: 返回所有权
3     let s1 = create_message();
4     println!("{}", s1);
5
6     // 方式2: 使用可变引用
7     let mut s2 = String::new();
8     build_message(&mut s2);
9     println!("{}", s2);
10 }
11
12 fn create_message() -> String {
13     String::from("Hello from function")
14 }
15
16 fn build_message(buffer: &mut String) {
17     buffer.push_str("Hello from mutable ref");
18 }
```



TIP

选择哪种方式取决于具体情况：

- 如果函数创建新数据，返回所有权
- 如果函数修改现有数据，使用可变引用
- 如果函数只读取数据，使用不可变引用

8 常见错误与解决

8.1 错误 1：使用已移动的值

```
1 fn main() {  
2     let s = String::from("hello");  
3     let t = s; // s移动给t  
4  
5     // println!("{}", s); // error[E0382]: borrow of moved value  
6 }
```



解决：

```
1 fn main() {  
2     let s = String::from("hello");  
3     let t = s.clone(); // 克隆，s仍然有效  
4  
5     println!("s={}, t={}", s, t);  
6 }
```



8.2 错误 2：同时存在可变和不可变引用

```
1 fn main() {  
2     let mut s = String::from("hello");  
3  
4     let r1 = &s;  
5     let r2 = &s;  
6     // let r3 = &mut s; // error[E0502]: cannot borrow as mutable  
7  
8     println!("{}", r1, r2);  
9     // 现在可以创建可变引用  
10    let r3 = &mut s;  
11    println!("{}", r3);  
12 }
```



解决：确保不可变引用不再使用后，再创建可变引用。

8.3 错误 3：返回局部变量的引用

```
1 // fn returns_reference() -> &String {  
2 //     let s = String::from("hello");  
3 //     &s // error[E0106]: missing lifetime specifier  
4 // }
```



解决:

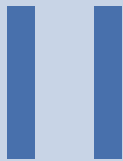
```
1 fn returns_owned() -> String {  
2     let s = String::from("hello");  
3     s // 返回所有权  
4 }
```

9 总结

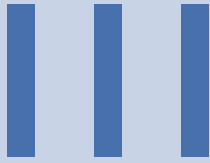
所有权和借用是 Rust 最核心的概念:

- **所有权**: 每个值都有唯一的所有者, 所有者离开作用域时值被释放
- **移动**: 赋值和传参时转移所有权, 避免双重释放
- **Copy**: 简单类型自动拷贝, 不涉及所有权转移
- **引用**: 允许借用值而不获取所有权
- **借用规则**:
 - 任意时刻要么有一个可变引用, 要么有多个不可变引用
 - 引用必须始终有效
- **切片**: 引用集合的一部分, 类型是 `&[T]` 或 `&str`

掌握所有权和借用是成为 Rust 程序员的关键。下一章将深入探讨结构体、枚举和模式匹配。



核心标准库与常用工具



高级特性与元编程

IV

并发编程



网络编程

网络编程



底层原理与性能调优